CANNESS 设计文档

CANN Operator Synchronization Optimization Test Harness

---

1. 项目定位

1.1 核心理念

CANNESS 是一个**面向 CANN 算子的垂域智能开发平台**，输入自然语言需求，通过多智能体协作闭环，完成算子**从无到有、从有到优**的全生命周期。

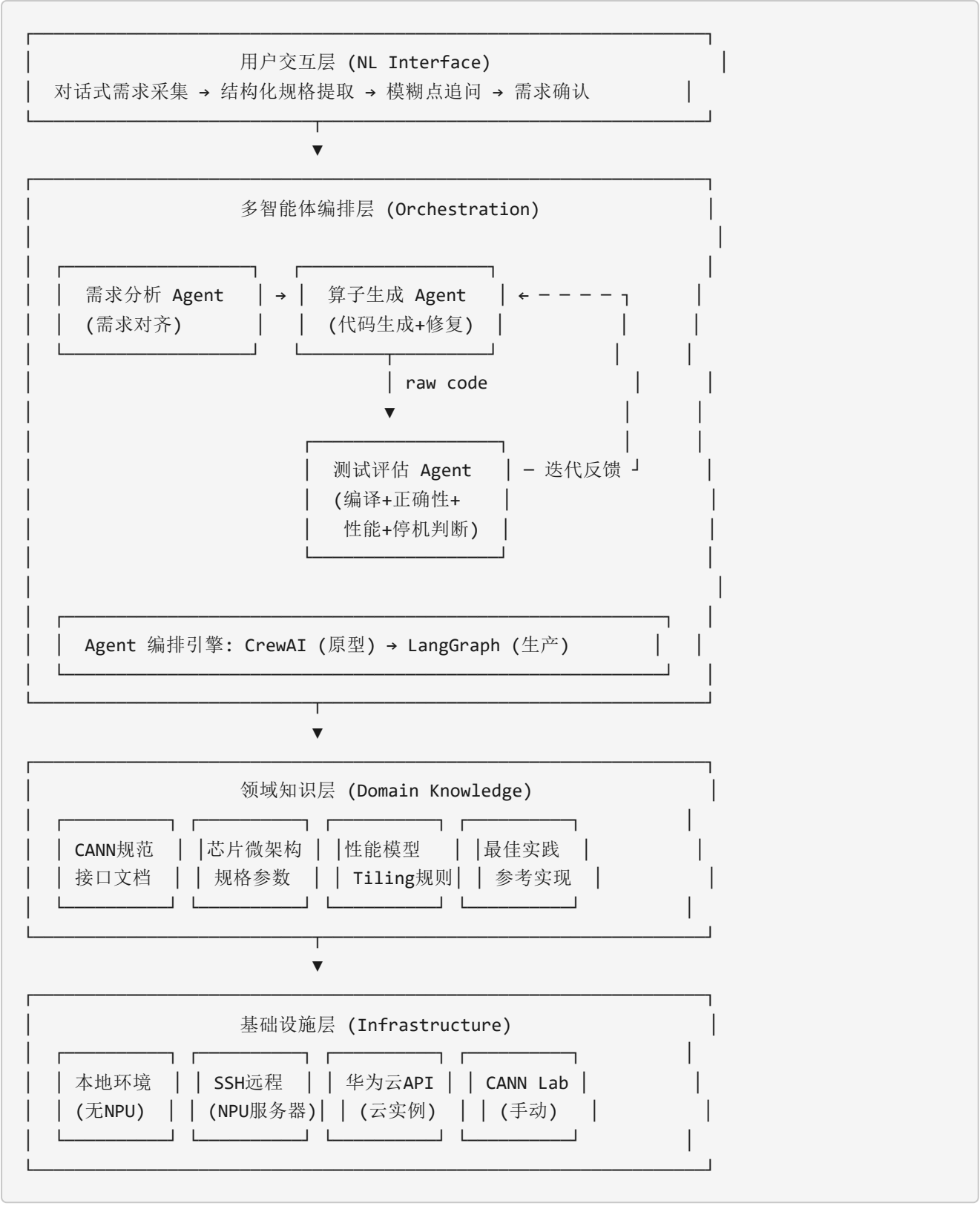
1.2 核心能力

阶段	能力	说明
从无到有	自然语言→算子代码	理解用户需求，对齐规格，生成四层算子代码
从有到优	测试反馈→迭代优化	编译、正确性、性能的闭环迭代，直至达标
横向拓展	算子融合协作	多算子融合分析（Phase 2）

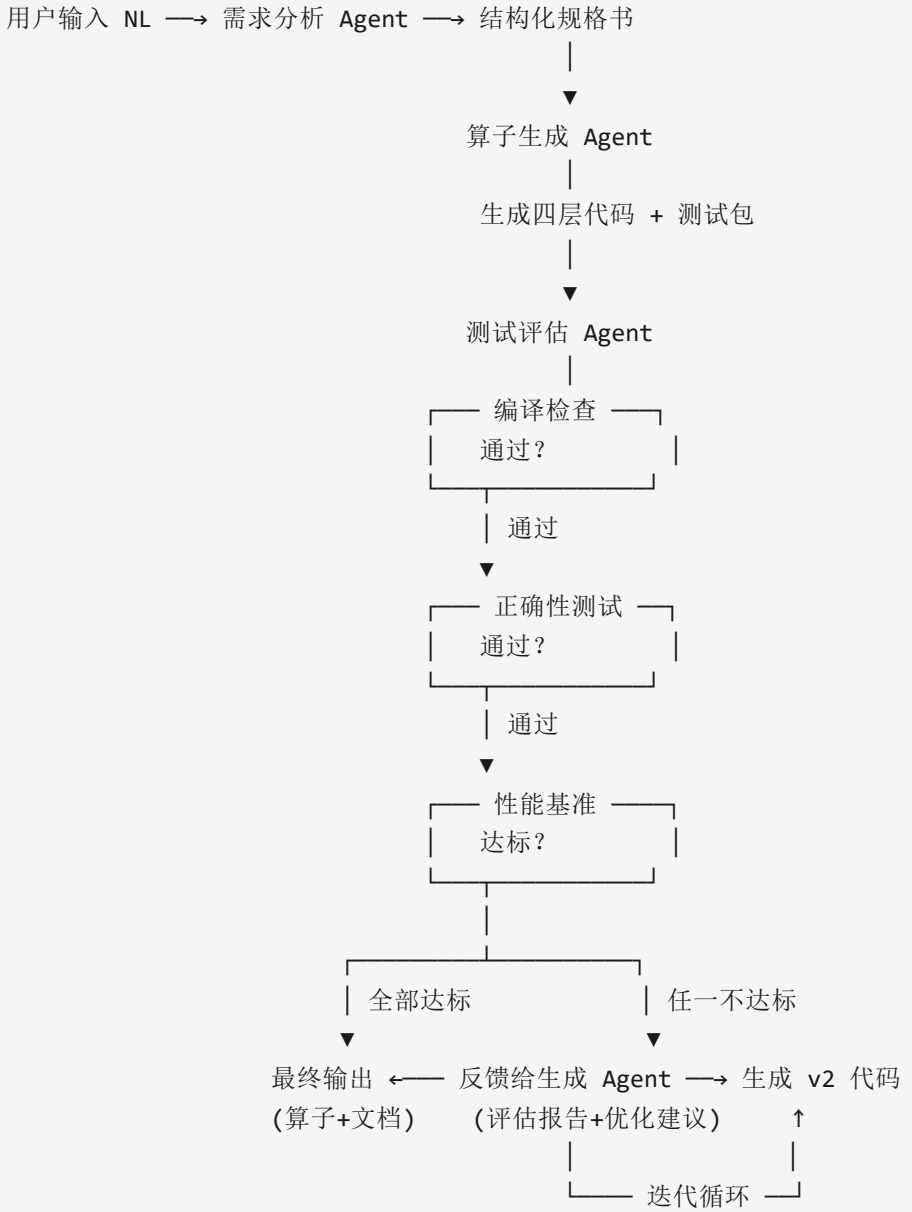
---

2. 整体架构

2.1 三层架构



2.2 闭环迭代流程



---

3. Agent 详细设计

3.1 需求分析 Agent

职责： 将用户自然语言需求转化为结构化的算子规格书

输入：

- 用户自然语言描述（如："我需要一个高效的 Conv2D，FP16，跑在 910B 上，batch_size=4"）

输出： 结构化规格书 (JSON)

```
{
  "operator_name": "Conv2D",
  "operator_type": "双目",
  "input_specs": [
    {
      "name": "input",
      "shape": ["N", "C", "H", "W"],
      "dtype": "float16",
      "constraints": { "N": 4, "C_in": 3, "H": [16, 1024], "W": [16, 1024] }
    },
    {
      "name": "weight",
      "shape": ["C_out", "C_in", "K_H", "K_W"],
      "dtype": "float16",
      "constraints": { "C_out": 64, "K_H": 3, "K_W": 3 }
    }
  ],
  "output_specs": [
    {
      "name": "output",
      "shape": ["N", "C_out", "H_out", "W_out"],
      "dtype": "float16"
    }
  ],
  "target_platform": {
    "chip": "Ascend910B",
    "soc_version": "Ascend910B1",
    "cann_version": "8.0.RC3"
  },
  "performance_target": {
    "metric": "kernel_time",
    "target_us": 150.0,
    "reference": "PyTorch CPU FP32 baseline * 0.8"
  },
  "tiling_hints": null
}
```

追问策略：

- 用户描述模糊时，识别缺失字段并追问
- 示例追问：“您期望的输入 shape 范围是多少？精度 FP16 还是 FP32？”

依赖知识库：

- CANN 算子接口规范
- 昇腾芯片规格表（910B vs 310P 参数）
- 算子输入输出模板

3.2 算子生成 Agent

职责： 根据规格书生成四层算子代码，并根据测试反馈迭代修复

输入：

- 结构化规格书
- 测试评估 Agent 返回的评估报告（迭代时）
- 知识库中的参考实现

输出：

- 四层算子代码：
  - `op_kernel/` — AscendC 核函数（NPU 执行）
  - `op_host/` — 算子定义、shape 推导、tiling 策略（CPU 执行）
  - `op_api/` — L2/L0 API 封装
  - `op_plugin/` — PyTorch 绑定
- 测试包（含 CMakeLists.txt、测试脚本、run.sh）

生成策略：

1. 从知识库检索最相似的参考实现作为模板
2. LLM 根据规格书 + 模板生成四层代码
3. 编译失败时，根据错误信息自动修复（调用编译器获取错误输出）
4. 性能不达标时，根据优化建议调整 tiling 参数或代码结构

依赖知识库：

- CANN 编程规范（TIK/TBE）
- 算子参考实现库（高频算子实例）
- Tiling 模板库
- 常见编译错误模式与修复方案

3.3 测试评估 Agent

职责： 编译检查、正确性验证、性能基准测试，生成统一评估报告，判断停机条件

输入：

- 算子生成 Agent 输出的测试包

输出：

- 评估报告 (JSON)
- 停机条件判定结果
- 优化建议（迭代时反馈给生成 Agent）

3.3.1 测试包结构

```
test_packages/
└── {operator}_{version}/
    ├── operator/
    │   ├── op_kernel/
    │   ├── op_host/
    │   ├── op_api/
    │   └── op_plugin/
    ├── tests/
    │   ├── test_correctness.py    # 正确性验证
    │   ├── test_boundary.py       # 边界测试
    │   └── test_performance.py    # 性能基准
    ├── reference/
    │   └── ref_impl.py            # PyTorch 参考实现
    ├── CMakeLists.txt
    ├── run.sh                     # 一键执行脚本
    ├── config.json                # 测试配置
    └── README.md                  # 操作说明
```

3.3.2 测试执行流程

```
Step 1: 编译
  cmake -DCMAKE_CXX_COMPILER=bisheng ..
  make -j4
  → 结果: 成功/失败 + 错误日志

Step 2: 正确性测试
  python tests/test_correctness.py
  → 生成随机输入 → 分别用参考实现和被测算子计算
  → 比对输出: torch.allclose(cann_out, ref_out, rtol=1e-3, atol=1e-5)
  → 结果: pass/fail + 逐 case 明细

Step 3: 边界测试
  python tests/test_boundary.py
  → 覆盖: 空输入、零维、极值、超大 shape、异常 dtype
  → 结果: pass/fail + 逐 case 明细

Step 4: 性能测试
  python tests/test_performance.py
  → 多次执行取中位数
  → 采集: kernel_time, throughput, memory_usage
  → 结果: 具体数值 + 与目标对比

Step 5: 汇总报告
  → 写入 results/results.json
```

3.3.3 评估报告格式

```
{
  "operator": "conv2d",
  "version": 3,
  "iteration": 3,
  "compile": {
    "success": true,
    "errors": [],
    "warnings": []
  },
  "correctness": {
    "pass": true,
    "total_cases": 12,
    "passed": 12,
    "failed_cases": [],
    "max_relative_error": 1.2e-6
  },
  "boundary": {
    "pass": true,
    "total_cases": 8,
    "passed": 8,
    "failed_cases": []
  },
  "performance": {
    "pass": true,
    "kernel_time_us": 125.3,
    "target_us": 150.0,
    "throughput": "达标",
    "memory_usage_mb": 256.0
  },
  "summary": {
    "all_pass": true,
    "failed_items": [],
    "optimization_suggestions": [
      "tiling 分块过小，建议增大 block_size 以提升 AIC 利用率",
      "当前使用默认流水线调度，可尝试 double buffer 优化"
    ]
  }
}
```

3.3.4 停机条件

条件	判定
编译通过	无 error，warning 可豁免
正确性通过	所有测试用例 pass，max_relative_error < 1e-3

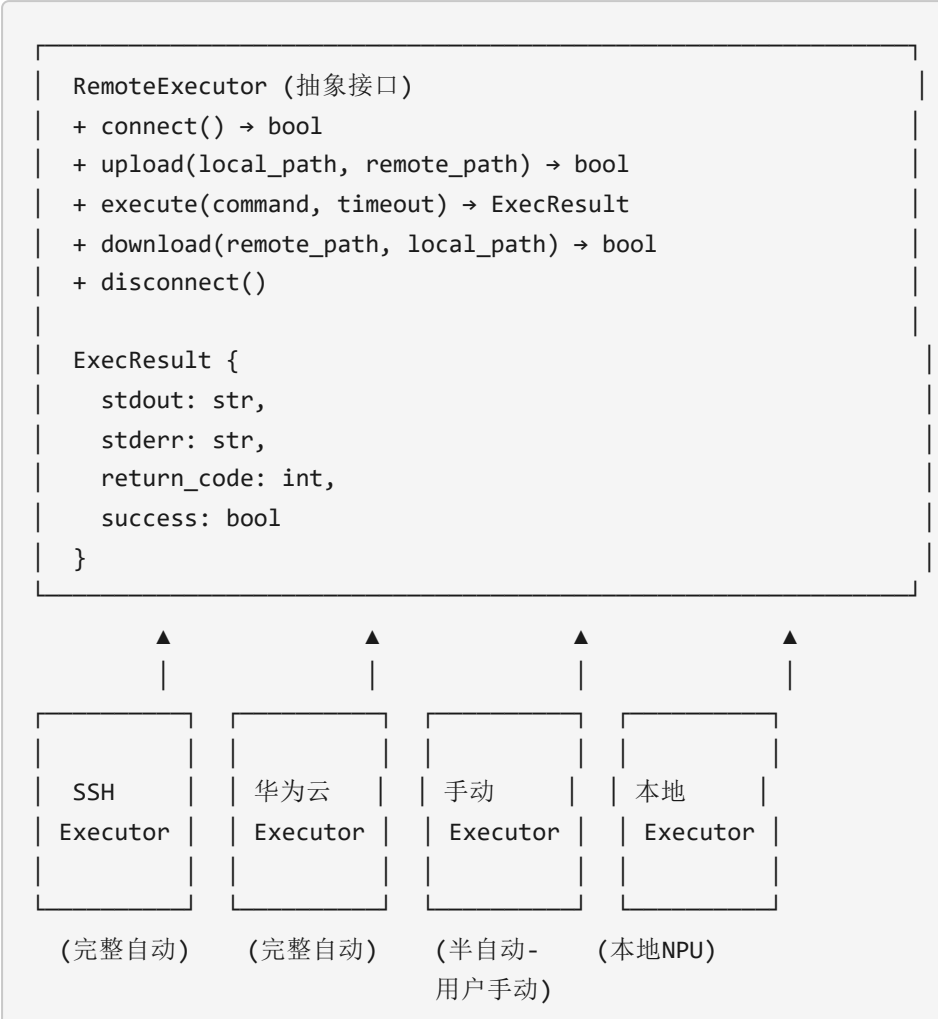
条件	判定
边界覆盖	所有边界用例 pass
性能达标	kernel_time <= target_us
最大迭代次数	默认 10 轮，可配置

全部满足 → 输出最终结果 任一不满足且未达最大迭代 → 反馈给生成 Agent

---

4. 远程执行架构

4.1 RemoteExecutor 抽象层



4.2 各实现说明

实现	自动化	连接方式	适用阶段
`SSHExecutor`	全自动	SSH + SCP	有固定 NPU 服务器

实现	自动化	连接方式	适用阶段
`CloudAPIExecutor`	全自动	华为云 REST API	云上测试
`ManualExecutor`	半自动	生成测试包，用户手动操作	Demo 阶段
`LocalExecutor`	全自动	本地 NPU 直连	开发者本地有 NPU

4.3 ManualExecutor 工作流

```
1. 测试评估 Agent 生成测试包至 test_packages/{op}_v{n}/
2. 输出提示信息:
   "请在 CANN Lab 执行以下操作:
    1. 上传 test_packages/{op}_v{n}/ 到 CANN Lab
    2. 执行 bash run.sh
    3. 将 results/ 目录下载到 feedback/{op}_v{n}/"
3. Agent 进入等待状态，轮询 feedback/{op}_v{n}/results.json
4. 检测到结果文件 → 自动解析 → 停机判断
```

---

5. 知识库设计

5.1 知识库结构

```
knowledge_base/
├── cann_specs/              # CANN 规范文档
│   ├── operator_api.md      # 算子接口规范
│   ├── tiling_guide.md      # Tiling 编程指南
│   └── ascendc_reference.md # AscendC API 参考
├── chip_specs/              # 芯片规格
│   ├── ascend_910b.json
│   └── ascend_310p.json
├── reference_ops/           # 参考算子实现
│   ├── add/
│   ├── conv2d/
│   ├── matmul/
│   └── relu/
├── tiling_templates/        # Tiling 策略模板
│   └── common_patterns.json
├── error_patterns/          # 常见编译错误与修复
│   └── compiler_errors.json
└── optimization_rules/      # 性能优化规则
    └── perf_rules.json
```

5.2 知识检索方式

- 算子生成 Agent 使用 **RAG (Retrieval-Augmented Generation)** 从知识库检索相关内容
- 参考实现作为 few-shot 示例注入 LLM 上下文
- Tiling 模板根据算子类型和芯片型号自动匹配

---

6. 技术栈

层级	技术选型	说明
编程语言	Python 3.12	主开发语言
Agent 编排	CrewAI (原型) / LangGraph (生产)	多智能体流程控制
LLM 接入	OpenAI API / DeepSeek-Coder / Qwen-Coder	代码生成与理解
向量数据库	Chroma / FAISS	知识库 RAG 检索
测试框架	pytest	测试执行
数据生成	NumPy	测试数据构造
远程执行	paramiko (SSH) / 华为云 SDK	远程 NPU 通信
配置管理	YAML + Pydantic	配置校验
报告输出	JSON + rich (CLI)	测试报告

---

7. 项目目录结构

```
CANNESS/
├── canness/                    # 核心源码
│   ├── agents/                 # Agent 定义
│   │   ├── __init__.py
│   │   ├── requirement_agent.py   # 需求分析 Agent
│   │   ├── generation_agent.py    # 算子生成 Agent
│   │   └── evaluation_agent.py    # 测试评估 Agent
│   ├── core/                   # 核心框架
│   │   ├── __init__.py
│   │   ├── orchestrator.py     # 编排引擎
│   │   ├── spec.py             # 规格书数据模型
│   │   └── report.py           # 评估报告数据模型
│   ├── executor/               # 远程执行器
│   │   ├── __init__.py
│   │   ├── base.py             # RemoteExecutor 抽象基类
│   │   ├── ssh_executor.py     # SSH 实现
│   │   ├── cloud_executor.py   # 华为云实现
│   │   └── manual_executor.py  # 手动实现
│   ├── knowledge/              # 知识库
│   │   ├── __init__.py
│   │   ├── loader.py           # 知识加载
│   │   └── retriever.py        # 知识检索 (RAG)
│   ├── test_package/           # 测试包生成
│   │   ├── __init__.py
│   │   ├── builder.py          # 测试包构建器
│   │   └── templates/          # 测试脚本模板
│   │       ├── test_correctness.py.j2
│   │       ├── test_boundary.py.j2
│   │       ├── test_performance.py.j2
│   │       └── run.sh.j2
│   └── utils/                  # 工具函数
│       ├── __init__.py
│       └── llm.py              # LLM 调用封装
├── config/                     # 配置
│   ├── default.yaml
│   └── agents.yaml             # Agent 配置
├── knowledge_base/             # 知识库文件
│   ├── cann_specs/
│   ├── chip_specs/
│   ├── reference_ops/
│   ├── tiling_templates/
│   ├── error_patterns/
│   └── optimization_rules/
├── test_packages/              # 生成的测试包输出目录
├── feedback/                   # 手动测试结果反馈目录
├── output/                     # 最终输出目录
│   └── operators/              # 最终算子代码
```

```
├── tests/                          # 框架自测
│   └── test_agents/
├── docs/                           # 文档
├── pyproject.toml
└── README.md
```

---

8. Phase 计划

Phase 1: 单算子闭环 (当前)

里程碑	内容	交付物
M1	需求分析 Agent	自然语言 → 结构化规格书
M2	算子生成 Agent + 知识库	规格书 → 四层算子代码
M3	测试评估 Agent	编译+正确性+性能测试闭环
M4	ManualExecutor + 完整迭代	手动测试反馈闭环
M5	SSHExecutor 接入	自动化远程执行

Phase 2: 算子融合协作 (后续)

里程碑	内容
M6	融合规划 Agent — 分析多算子融合机会
M7	融合算子生成与测试
M8	多算子流水线协作优化

---

9. 数据流总览

```
用户: "给一个 Conv2D, FP16, 910B, 性能要快"
  │
  ▼
[需求分析 Agent]
  └ 追问: "输入 shape 范围? 精度要求?"
  └ 输出: 规格书 JSON
  │
  ▼
[算子生成 Agent]
  └ 检索知识库 → 参考实现
  └ LLM 生成四层代码
  └ 构建测试包 → test_packages/conv2d_v1/
  │
  ▼
[测试评估 Agent]
  └ [ManualExecutor] 提示用户上传到 CANN Lab
  └ [等待 feedback/conv2d_v1/results.json]
  └ 检测到结果 → 解析
  └ 停机判断
      ├ 全部达标 → 输出到 output/operators/conv2d/
      └ 未达标 → 生成优化建议 → 送回生成 Agent
                    └ 生成 v2 → 测试 → ...
```

---

10. 设计原则

1. **Agent 间松耦合** — 通过结构化数据（规格书 JSON、评估报告 JSON）通信，不直接依赖内部实现
2. **执行器可插拔** — RemoteExecutor 策略模式，切换后端不影响 Agent 逻辑
3. **测试包自包含** — 测试包不依赖 CANNESS 框架，任何有 NPU 的环境都能跑
4. **反馈驱动迭代** — 每次测试结果驱动下一轮优化，形成闭环
5. **渐进式交付** — Phase 1 单算子 → Phase 2 融合算子，逐步扩展